# Glaukopis: Constructing a 32B CTI LLM Using Chained Instruction Fine-Tuning

Peter J. Worth Jr.[1,3,*], Ionut Cardei[1,3], Salman Ahmad[4], Muhammad Mamoon Khan[4]
[1]*Dept. of Electrical Engineering and Computer Science, Florida Atlantic University*, Boca Raton, FL, USA
[3]*Athena Security Group, LLC*, USA [4]*RevolveAI*
[*]Corresponding author: pworthjr@athenasecuritygrp.com

***Abstract*—Cyber threat intelligence (CTI) workflows in regulated security operations cannot in general rely on frontier proprietary models due to cost, latency, and data-handling constraints, while generic foundation models lack the schema fidelity, applied reasoning, and current-threat awareness those workflows require. This paper introduces Glaukopis, a methodology and working system for producing on-premises 32B-scale CTI expert language models via *chained instruction fine-tuning* (chained IFT): an IFT pipeline in which each Alpaca-format (instruction, input, output) triple is materialized by a typed query against an authoritative knowledge graph—so every row is graph-grounded rather than synthesized—and in which the supervised pass is decomposed into a chain of stages with distinct dataset shards and hyperparameter regimes. Glaukopis has three components: Ariadne, a Neo4j substrate ingesting twelve public CTI sources into ∼280K nodes and ∼480K edges; Sophia, a graph-template generator that compiles declarative CTI reasoning patterns into IFT triples consistent with the graph; and a four-stage IFT chain (Core, +TAA, CSE, Recalibrate-32B). The v21 ship checkpoint reaches 66.3 average CTI benchmark score (vs. 59.0 for its Qwen-2.5-32B-Instruct base), approaching DeepSeek-V4-Pro (70.4), Gemini-3-Flash-Preview (69.8), and GPT-5.5 (72.9) at approximately two orders of magnitude lower per-evaluation cost.**

***Index Terms*—instruction fine-tuning, chained IFT, supervised fine-tuning, small language models, cyber threat intelligence, CTI knowledge graphs, Neo4j, MITRE ATT&CK, AthenaBench**

## I. Introduction

Large language models (LLMs) are now embedded throughout the cyber security workflow: they summarize threat reports, map indicators to MITRE ATT&CK, triage vulnerabilities, draft detection rules, and answer analyst questions inside security operations centers (SOCs). Surveys of security LLMs consistently identify the same residual failure modes: hallucinations on schema-bearing fields, shallow pattern matching where multi-hop reasoning is required, and a structural inability to operate on threat information more recent than the base model's pretraining cutoff [1], [2].

**The operational dilemma.** The industry response has been an awkward two-way split. Generic foundation models are inexpensive and easy to host but underperform on CTI-specific tasks—schema fidelity (CVE/CWE/ATT&CK identifiers, CVSS vectors), applied reasoning (root-cause mapping, severity estimation, threat-actor attribution), and current-threat awareness are all weak. Frontier proprietary models close much of the gap, but their cost, latency, and data-handling characteristics are incompatible with most regulated security operations: SOCs supporting healthcare, financial, or government workloads cannot, in general, send raw threat telemetry to a third-party API, and the per-evaluation cost of a frontier model across a realistic CTI workload is prohibitive at analyst-team scale.

**Chained instruction fine-tuning.** A natural alternative is a small (sub-32B) model post-trained on CTI material from authoritative public sources and deployable on-premises. The approach pursued here, *chained instruction fine-tuning* (chained IFT), refines the standard IFT pipeline [3]–[5] along two axes: (i) on the *data* side, every Alpaca-format triple is materialized by a typed query against an authoritative knowledge graph rather than authored or LLM-synthesized, so each row is grounded in named graph nodes traceable to public-source natural keys; and (ii) on the *training* side, the supervised pass is decomposed into a chain of stages with distinct shards and hyperparameter regimes, so capabilities accumulate compositionally and per-axis regressions are attributable to specific stages. The closest neighbors are IBM's CyberPal.AI and SecKnowledge, which couple expert-curated instruction schemas with chain-of-thought enrichment [1], [6], and SEvenLLM and CyberBench, which adapt smaller open models through hand-crafted or LLM-synthesized corpora [2], [7].

**The Glaukopis system.** Glaukopis is the working instantiation of chained IFT, with three named components: **Ariadne** (Section IV), a Neo4j substrate ingesting twelve public CTI sources into ∼280K nodes and ∼480K edges, refreshed daily; **Sophia** (Section IV), a graph-template generator that compiles declarative CTI reasoning patterns into Alpaca triples whose outputs are computed directly from Cypher queries; and a **four-stage IFT chain** (Section V)—Core, +TAA, CSE, Recalibrate-32B—each with its own shard, hyperparameter regime, and capability outcome (Figure 1).

**Headline result.** The v21 ship checkpoint, `asg-ai/athena-cti-sft-qwen25-32b-v21-recal-32b`, reaches an average CTI-benchmark score of 66.3 across AthenaBench, CyberSOCEval, and CyberMetric—7.3 points over its `Qwen/Qwen2.5-32B-Instruct` base (59.0), without changing parameter count or runtime footprint. It exceeds Gemini-2.5-Flash (63.4), runs essentially even with DeepSeek-V3.2-Exp (66.0), and sits within 4–7 points of the leading fron-

This research was funded by Athena Security Group, LLC.

statements like this must be more refined and grounded by published evidence or experiments.
"Are all weak" is not needed, because of "underperform"
Please give some references.

would help some references

the Aplaca format is not relevant; I would not refer to it at this point

... and edges

IFT triples have no outputs; they are the outputs from template -> Cypher queries

is it useful to list the full version or file name?
To keep readability, you may just insert a reference that points to the version name and URL in the References section

tier references—DeepSeek-V4-Pro (70.4), Gemini-3-Flash-Preview (69.8), GPT-5.5 (72.9)—at roughly two orders of magnitude lower per-evaluation cost (Section VI).

**Contributions.** (1) We introduce chained IFT as a refinement of standard IFT along graph-grounded data construction and multi-stage scheduling, and locate it within the FLAN/InstructGPT/FLAN-T5 lineage and the CyberPal.AI/SecKnowledge line. (2) We describe Ariadne and Sophia, a complete, auditable training-data pipeline in which every row maps to specific public-source identifiers. (3) We specify and ship a four-stage chained-IFT recipe with full per-stage hyperparameters, and report a strict-reproducibility experiment isolating data-build non-determinism from recipe drift. (4) We report a benchmark portfolio (AthenaBench, CyberSOCEval, CyberMetric, MMLU-Pro) with a cost analysis treating per-evaluation dollar cost as a first-class metric. (5) We release source and checkpoints at `https://github.com/Athena-Software-Group/Glaukopis` [8].

## II. Background and Related Work

**IFT foundations.** Instruction fine-tuning (IFT) adapts a pretrained LLM on labeled (instruction, optional context, target) data, maximizing log-likelihood of the reference output [3], [4]; it is one instantiation of supervised fine-tuning (SFT), and we use "IFT" following the cyber security literature. FLAN [3] established that instruction-following is a *learnable* capability rather than a pure emergent of scale. InstructGPT [4] introduced the staged SFT→preference→RLHF pipeline and showed that small instruction-tuned models can be preferred over much larger unaligned ones—the load-bearing argument for the small-model strategy here. FLAN-T5 [5] scaled the instruction corpus to thousands of tasks and showed that instruction-data diversity and structure matter at least as much as parameter count, licensing the Glaukopis choice to invest in instruction-data structure at a fixed 32B budget.

**Chained IFT.** A recurring choice in modern post-training is to compose several IFT stages into a chain rather than a single monolithic tune: the output of each stage initializes the next, capabilities accumulate compositionally, and individual stages can be re-run when their data changes. Separating broad domain absorption from precise task behavior reduces destructive interference [4], [9]; reasoning supervision is more effective layered on top of a model that already commands the domain vocabulary [6], [10]; and staging makes pipelines incrementally refreshable, which matters where source data turns over weekly. Glaukopis adopts a four-stage chain (Core, +TAA, CSE, Recalibrate-32B); the CyberPal.AI/SecKnowledge line operationalizes a related staging discipline on a different data pipeline [1], [6].

**Graph-grounded IFT.** The second refinement is on the data side: every triple is materialized by a typed query against an authoritative knowledge graph. This converges three lines: schema- and grammar-constrained generation (target outputs computed by executing queries against a reference database, not authored); knowledge-graph-grounded QA, where a typed traversal returns a subgraph rendered into a QA pair (CTI instantiations include [11]–[15]); and expert-schema-driven IFT for cyber security [1], [6]. Glaukopis combines all three: the Ariadne schema defines the typed substrate, every Sophia row is materialized by a Cypher query against the populated graph, and the driving templates are expert-authored CTI reasoning patterns.

define abbrev.
Is it Q&A?

**IFT and SLMs for cyber security.** Cyber security imposes heterogeneous schemas (CVE, CWE, CAPEC, ATT&CK), evidence-grounded structured outputs, and rapid concept drift. SEvenLLM [7] builds a bilingual instruction dataset and benchmark from incident reports; CyberBench [2] provides cyber-specific instruction tasks and an evaluation suite. The closest neighbor is IBM's CyberPal.AI [1], which mines ATT&CK, Sigma/SIEM rules, emulation plans, and threat reports into expert-defined instruction schemas; the SecKnowledge 2.0 follow-on [6] adds chain-of-thought enrichment, training 4–20B CTI-expert SLMs that rival larger general models. Glaukopis differs in three ways: every row is materialized from a typed CTI graph rather than synthesized; the SFT pass is a four-stage chain with explicit per-axis recovery; and the target is 32B—the smallest deployment-feasible scale at which the specialization/general-capability trade-off is acceptable for our workloads (Section VI). Such on-premises, air-gapped, low-latency deployment is exactly the regime in which well-targeted SLMs are most attractive [6].

## III. Cyber Security LLM Benchmarks

General benchmarks (GLUE, MMLU, BIG-Bench) measure broad language understanding but were not designed to probe security workflows: they lack the structured schemas, evidence-grounded reasoning, and threat-current content that define CTI tasks, and—because MITRE, NVD, and public threat reports are heavily represented in pretraining corpora—static suites risk measuring recall of training data rather than reasoning [16]. The past two years have produced cyber-specific benchmarks spanning four overlapping families: knowledge (CyberMetric [17], SecEval [18]), secure-coding/offensive capability (Purple Llama CyberSecEval [19], [20], WMDP [21]), applied CTI and SOC reasoning (CTIBench [22], SEvenLLM-Bench [7], CyberSOCEval [23]), and dynamic/live-source suites (AthenaBench [16]).

No single benchmark suffices, so Glaukopis is evaluated against a portfolio: **AthenaBench** (a dynamic CTI suite drawing from live MITRE/NVD/CISA sources, with six tasks—CKT, RCM, VSP, TAA, RMS, ATE—that target staleness and contamination directly [16]); **CyberSOCEval** malware-analysis and threat-intel-reasoning suites (operational SOC reasoning over free-form narrative input [23]); **CyberMetric** at the 2K and 10K tiers (RAG-built, expert-validated MCQ giving statistically robust factual coverage [17]); and **MMLU-Pro** as a decoupled general-intelligence reference. AthenaBench is the training counterpart to Sophia/Ariadne: both derive from the same public sources but via independent pipelines, and Sophia's version/timestamp filters let training recency be aligned to AthenaBench's dynamic horizon to mitigate leakage [16], [24].

would help to define these tasks. Don't assume reviewers are familiar with CTI LLMs.

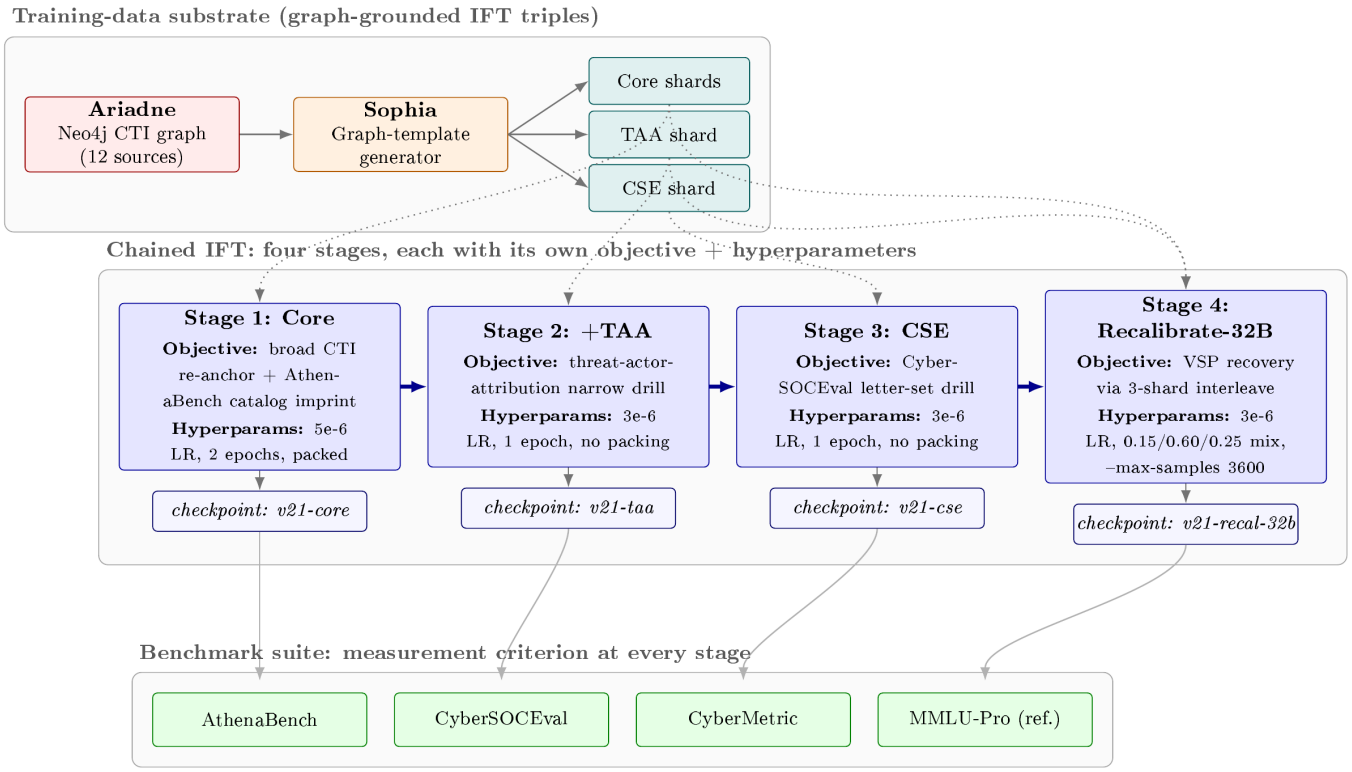


Fig. 1. **The chained-IFT pattern.** Sophia compiles typed graph queries against Ariadne into Alpaca-format IFT triples, sharded into Core, TAA, and CSE manifests. The four-stage chain consumes these in order: each stage has its own objective (broad CTI re-anchor for Core, narrow drills for TAA and CSE, recalibration for Stage 4) and hyperparameters. Each stage pushes a cumulative checkpoint (`v21-core`, `v21-taa`, `v21-cse`, `v21-recal-32b`), each independently evaluable against the AthenaBench / CyberSOCEval / CyberMetric / MMLU-Pro suite, supporting per-stage ablation in addition to producing a single ship checkpoint.

## IV. Ariadne and Sophia: The Training-Data Pipeline

**Ariadne: the CTI knowledge graph.** Cyber threat intelligence is the collection, analysis, and dissemination of information about threats and adversaries to inform defensive decisions [25]. Ariadne is the populated Neo4j substrate that all downstream components (template generation, SFT, evaluation harness) consume as their single source of CTI facts; every fact in any training example must first exist as a node or edge in Ariadne, making the graph both the corpus's data layer and the harness's reference oracle. It ingests twelve public CTI sources spanning offensive technique catalogues (MITRE ATT&CK [26], ENGAGE), attack-pattern and weakness taxonomies (CAPEC [27], CWE [28]), defensive countermeasures (D3FEND, pinned to v1.4.0), vulnerability records (CVE 5.0 [29], NVD CPE/CVSS), exploitation signals (CISA KEV, FIRST EPSS), detection content (Sigma), and weaponized-exploit evidence (ExploitDB, PoC-in-GitHub). The bounded catalogues (ATT&CK, ENGAGE, CAPEC, CWE, KEV, D3FEND, Sigma) load in full; CVE, NVD, EPSS, ExploitDB, and PoC-in-GitHub are scoped to 2024 onward, keeping the graph at ∼280K nodes and ∼480K edges—large enough for multi-hop cross-schema reasoning, small enough for practical single-instance query latency.

The populator is idempotent (`MERGE` against per-node uniqueness constraints on `stix_id`, CVE/CWE/D3FEND IDs, etc.), so re-runs never duplicate. Three refresh policies apply: *always-refresh* (EPSS, current-year NVD, both daily), *skip-if-cached HTTPS* (CAPEC, CWE, KEV, D3FEND), and *skip-if-cached Git* (ATT&CK, ENGAGE, CVE, Sigma, ExploitDB, PoC-in-GitHub); production runs a nightly cron that picks up daily EPSS/NVD churn while leaving static catalogues untouched. Construction proceeds in a fixed dependency order—framework nodes first, then the vulnerability surface, then the weaponized-exploit layer—so cross-framework edges (CVE→CWE root-cause, CWE→CAPEC→ATT&CK pivots, ATT&CK→Mitigation, D3FEND↔ATT&CK) always have both endpoints present. Every SFT row is traceable to specific graph identifiers, which resolve to public-source rows by their natural keys—the basis of the chained-IFT auditability claim.

**Sophia: graph-template-driven generation.** Sophia (`tmpl_gen`) compiles declarative CTI reasoning patterns into Alpaca triples by issuing Cypher queries against Ariadne and binding returned nodes, properties, and relationships into (instruction, input, output) records [24]. Because every template resolves to typed traversals over named-key constraints, each triple is grounded in specific elements

citable by natural key (e.g., `CVE-2024-12345`, `T1059.003`, `CAPEC-66`). A three-step wrapper (`make_dataset.sh`) drives it: `docx2json` extracts templates from `.docx`/JSON authoring formats; `tmpl2triples` opens a Bolt connection, issues the implied queries, and emits per-template triples with a `_results-report.json` summary of failures; `triples2alpaca` merges them into one dataset. A representative template binds a node to an alias and emits its properties:

```
M.1 Instruction: You are a cyber security
expert...
Question: Explain ATT&CK technique
{ap:attack-pattern.name} ({ap.id}) and how it
is used.
Answer: Technique {ap.name} ({ap.id}) is described
as: {ap.description}. Commonly observed across
{ap.x_mitre_platforms}.
```

The full language adds relationship traversal (`{v.rel>Type.prop}`), filters with any/all semantics on list-valued properties, list-comprehension expansions, invisible binding sections, and source-disambiguating prefixes (`attck#attack-pattern` vs. `capec#attack-pattern`) [24]. Its scientific advantages over LLM-authored instruction sets are *ground-truth faithfulness* (answers computed from an authoritative graph), *explicit coverage control*, *rich hard negatives* (mitigations that do *not* address a technique, etc.), *auditability* (each row re-derivable by re-running its Cypher), and *incremental refresh* via version/date filters [16], [24]. Conceptually, where CyberPal synthesizes instructions from unstructured sources using LLMs and experts [1], [6], Sophia uses a declarative template language over a typed graph to generate strongly grounded, precisely controllable triples. Figure 2 shows the end-to-end flow from the upstream public CTI sources through the Sophia generator and the v21 curation pipeline to the Alpaca-format IFT shards consumed by the chained-IFT stages.

## V. Glaukopis: The Chained-IFT Pipeline

Glaukopis produces the Athena-CTI-LLM family via template-based chained IFT: Ariadne drives Sophia (Section IV); Sophia emits IFT triples sharded into purpose-built datasets; those feed a four-stage chain; and each resulting checkpoint is evaluated against the portfolio of Section III.

**The v21 vintage.** The active vintage is **v21** (template directory `tmpl_gen/templates/05182026/`), which projects Ariadne through three template manifests—Core, TAA, and CSE—into four production training shards plus three matched validation shards. All shards are template-baked and therefore architecture-independent: one set of seven JSON files feeds every base-model port. v21 is a strict reproducibility experiment forked from v18.1: the manifests, row-count gates, and per-stage count limits are `shasum`-verified byte-identical to v18.1, with only dataset names and Hugging Face push targets changed. The sign-off gate was that the post-Core checkpoint land within $\pm 1.5$ points of the v18.1 Core peak on every axis; Total Score landed inside the band but two sub-scores (ATE, RCM) drifted, with the same shape seen in the intermediate v19/v20 vintages. Since the recipe-side inputs are byte-identical, the most parsimonious explanation is data-build non-determinism in the substrate sampler, dedup tiebreaks, and shuffle seeding inside `make_dataset.sh`—making seed-pinning the next-vintage direction. Carrying byte-identical templates and gates forward across vintages isolates recipe drift from data-layer drift as separable failure modes, which is itself a contribution.

The same recipe ports byte-identically across five base architectures (Table I); any axis-level differences are therefore attributable to base-model differences rather than training-data drift. The Qwen3-MoE port additionally exercises a thinking-on/off A/B at serve time.

**The four-stage SFT chain.** Each stage runs standard SFT on next-token cross-entropy; the chain consumes seven Sophia shards. **Core-A** is a broad re-anchor mix (knowledge-base, MCQ, TAA, SOC, CyberMetric-style, mitigation, yes/no families); **Core-B** is an AthenaBench catalog drill (RMS, ATE, VSP, RCM); **TAA** is a narrow TAA-Classic drill; **CSE** is a CyberSOCEval letter-set drill; three matched validation shards hold out 50 rows per shortname.

*Stage 1 (Core)* runs two phases, pushing only the final merged checkpoint. Phase A re-establishes broad instruction-following on Core-A plus a general mix (Tülu-3 SFT, Alpaca-English) at seq-len 8 192, packing on, LR $1 \times 10^{-5}$, effective batch 16. Phase B drills Core-B at seq-len 16 384, packing off (so long structured outputs are not truncated), LR $5 \times 10^{-6}$, effective batch 8, imprinting precise AthenaBench task structure [5], [9], [16]. *Stage 2 (+TAA)* trains the Core checkpoint on TAA alone (seq-len 4 096, packing on, LR $5 \times 10^{-6}$, batch 16), hardening TAA-Classic without disturbing catalog skills. *Stage 3 (CSE)* trains the +TAA checkpoint on CSE at the same settings, targeting the CyberSOCEval suites [23]; this reliably trades $\sim$10 points of VSP for the CyberSOCEval lift. *Stage 4 (Recalibrate, ship)* is an off-plan three-shard interleaved touch-up over Core-A/Core-B/TAA that recovers the VSP regression without losing Stage 3 gains. At 14B a single recipe (LR $1 \times 10^{-6}$, 0.25/0.40/0.35 mix, `--max-samples2400`) cleanly recovers VSP; at 32B the same recipe sits at the `adamw_8bit` noise floor and drifts VSP the wrong way, so it is re-tuned (LR $3 \times 10^{-6}$, Phase-B-heavy 0.15/0.60/0.25 mix, `--max-samples3600`) via LlamaFactory's `interleave_under` strategy. All stages share bf16, DeepSpeed ZeRO-3, the `adamw_8bit` optimizer (mandatory at 32B), Liger kernels, and gradient checkpointing; the canonical target is 8×H100-80GB, with effective batch held constant across topologies by auto-scaling gradient accumulation.

The four-stage structure is the load-bearing element: Stages 1A/1B separate broad absorption from structured-task imprinting, Stages 2/3 drill specific axes without re-perturbing prior skills, and Stage 4 recovers regressed axes through controlled interleaving. Because every stage pushes a checkpoint and consumes a specific shard, any axis-level regression is attributable to a specific stage and addressable by re-running only the affected stages.

Not sure if script names are important to be in the paper since we may change things in future versions.

it would help to add somewhere in the intro and here again with mode details the size of the training set, maybe broken down per stage, expressed in triple count and tokens or equivalent

more useful would be an example with a 2 or 3 node subpath

is that learning rate? What is seq-len? Agents tend to create these abbreviations but the paper must be crystal clear.

14B parameter model?

do these details of 21 vs. 18 matter to the reader? Readers have no reference point. They may complicate the paper without much payoff.

unclear what these shards are

this is the very important; can you include test results to back up these conclusions?

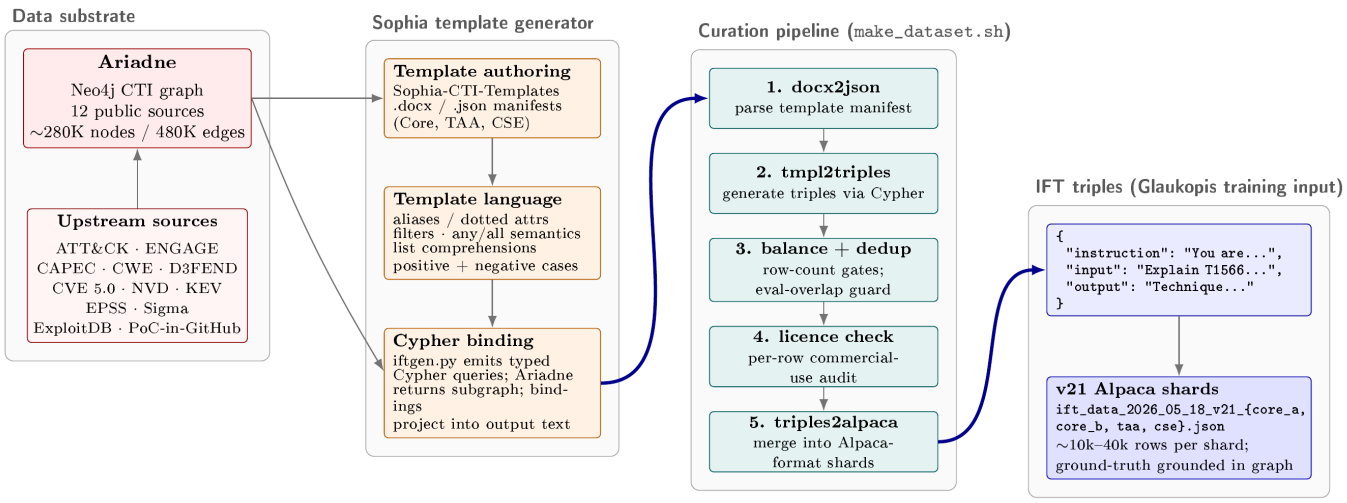


Fig. 2. **Sophia template generation and v21 data flow.** The upstream public CTI sources (left) are ingested into the Ariadne Neo4j graph. Sophia (centre-left) authors templates in three artefacts—template manifests (.docx/.json), the template language, and the Cypher binding step in `iftgen.py`—each of which compiles graph queries into natural-language output. The v21 curation pipeline (`make_dataset.sh`, centre-right) runs five steps in order: parse the template manifest, generate triples via Cypher, balance and dedup the corpus against the row-count gates and the eval-overlap guard (Section VII), check per-row licence compliance, and merge the per-template files into Alpaca-format shards. The output (right) is the set of Alpaca-format IFT shard files (`ift_data_2026_05_18_v21_{core_a, core_b, taa, cse}.json`) consumed directly by the chained-IFT training stages (Section V).

I really am not sure if inclusion of actual version and script names help the reader. These make the paper more stuffy, since they are not use elsewhere and are not used to differentiate or refer to things. These seem impl details, taking space.

TABLE I
BASELINE INSTRUCT MODELS TARGETED BY THE V21 ARCHITECTURE PORTS. THE DENSE QWEN-2.5-32B-INSTRUCT IS THE CANONICAL STAGE-0 BASE FOR THE SHIP CHECKPOINT REPORTED IN SECTION VI; THE OTHER FOUR ARE CROSS-ARCHITECTURE PORTS OF THE SAME V21 CHAIN.

| Canonical HF identifier | Family | Architecture | Role in v21 |
|---|---|---|---|
| `Qwen/Qwen2.5-32B-Instruct` | Qwen 2.5 | 32B dense | Canonical Stage-0 base (ship) |
| `Qwen/Qwen2.5-14B-Instruct` | Qwen 2.5 | 14B dense | Cross-architecture port |
| `Qwen/Qwen3-30B-A3B-Thinking-2507` | Qwen 3 Thinking | 30B MoE (A3B) | MoE / thinking-mode port |
| `meta-llama/Meta-Llama-3.1-8B-Instruct` | Llama 3.1 | 8B dense | Cross-architecture port |
| `fdtn-ai/Foundation-Sec-8B-Instruct` | Foundation-Sec | 8B dense | Security-specialized base (Cisco) |

citation?

**Evaluation harness.** Each checkpoint is served via vLLM (OpenAI-compatible) and scored on AthenaBench (six CTI tasks), CyberSOCEval (malware + threat-intel), CyberMetric (2K, 10K), and MMLU-Pro; the same alias-keyed harness scores public baselines through hosted providers. Per-task token and wall-clock usage is captured so cost is reported alongside accuracy. The stage→suite mapping is deliberate: Core-B exercises RMS/ATE/VSP/RCM, Stage 2 exercises TAA, Stage 3 exercises CyberSOCEval, and Core-A exercises CyberMetric and the MCQ axis; MMLU-Pro is run separately as a forgetting diagnostic, not an optimization target.

## VI. RESULTS

We evaluate the v21 vintage against the portfolio of Section III. All numbers come from a single harness run (sweep dated 2026-05-24), using the same vLLM stack for local checkpoints and the same hosted endpoints for public baselines.

**Three averaging schemes.** Because the CTI benchmarks differ in internal structure (AthenaBench has six tasks, CyberSOCEval two, CyberMetric two tiers), the choice of aggregation visibly changes the leaderboard, so we report three. *Avg CTI Benchmark* is the unweighted mean of three per-benchmark summaries (each benchmark equal weight)—our cross-benchmark metric, closest to published leaderboards. *Avg CTI Test* is the unweighted mean across the ten individual CTI tests, the most apples-to-apples cross-architecture view. *Avg Total (w/MMLU-Pro)* extends the per-test average with MMLU-Pro to quantify how much general capability each run trades away—a *general-knowledge erosion* diagnostic, not an optimization target. Cost is measured uniformly: published per-1K-token rates for hosted checkpoints, and wall-clock at $2.50/GPU-hour on a 2×H100 host for local vLLM checkpoints.

**Headline comparison.** Table II compares the v21 ship checkpoint against its base and six frontier references. Three observations: (1) chained SFT adds **7.3 points** of Avg CTI Benchmark over Qwen-2.5-32B-Instruct (59.0 → 66.3) and 13.3 on Avg CTI Test (49.6 → 62.9), with the largest single-axis lift on AthenaBench combined (44.2 → 66.5, +22.3). (2)

The ship sits within 4–7 points of the leading frontier models (GPT-5.5 72.9, DeepSeek-V4-Pro 70.4, Gemini-3-Flash-Preview 69.8), exceeds Gemini-2.5-Flash (63.4), and runs even with DeepSeek-V3.2-Exp (66.0) despite the parameter gap. (3) General-knowledge erosion is real but graded and not unique to Glaukopis: MMLU-Pro drops $69.0 \rightarrow 57.4$ ($-11.6$), reflecting the known cost of specialization; we report it as a diagnostic rather than optimizing against it.

The two Stage-4 variants share the same Stage-3 parent and differ only in recipe. On headline aggregates they are within 0.5 points; the deciding factor is per-axis VSP recovery: `v21-recal-32b` restores VSP to 77.8 vs. `v21-recalibrate`'s 75.7, which is precisely what Stage 4 is for at 32B. `v21-recal-32b` is therefore the ship checkpoint.

**Per-stage ablation.** Because the chain pushes a checkpoint after every stage, per-stage contributions are directly measurable (Table III). Stage 1 (Core) does most of the catalog work—ATE $18.2 \rightarrow 67.2$, RMS $7.6 \rightarrow 63.8$, RCM $59.5 \rightarrow 69.8$. Stage 2 introduces Canonical-TAA capability (TAA Canonical $2.0/18.4 \rightarrow 36.8$). Stage 3 lifts CyberSOCEval ($31.1 \rightarrow 42.0$) at the expected VSP cost ($83.1 \rightarrow 78.9$). Stage 4 is recovery, not novel capability: it re-aligns VSP (77.8) and stabilizes CyberSOCEval (42.6) rather than developing new capability.

**Cross-architecture ports.** The same recipe ports byte-identically to four other bases (Table IV). Recipe transfer scales with base size: the 14B and 32B dense Qwen ports lift +13.5/+13.3 on Avg CTI Test, Llama-3.1-8B +7.3, the Qwen3-MoE port +6.0, and Foundation-Sec-8B only +0.4 (already a security-specialized base). The Qwen3-MoE port leads on MMLU-Pro among SFT'd checkpoints (53.0) but is more expensive to serve and loses on the CTI averages; the dense Qwen-2.5-32B port is the most CTI-capable and most efficient ship. MMLU-Pro regression is largest at smaller scales (8B: 17.6–20.6; 14B: 13.9; 32B: 11.6; MoE: 24.6).

**Cost.** On a full-suite pass, the v21 ship costs $2.83 (Table V). The most expensive proprietary model, Gemini-3-Flash-Preview, costs $1,154.13 (408×); GPT-5.5 $508.13 (180×); Gemini-2.5-Flash—which the ship beats on every CTI average—$166.24 (59×); DeepSeek-V4-Pro $126.40 (45×). The cheapest API-priced references (DeepSeek-V3 family, $11–14) are still 4–5× the locally-served ship despite far larger parameter counts. Normalized by capability, the ship sits at $0.043 per Avg-CTI-Bench point—an order of magnitude below the cheapest API frontier reference ($0.17) and ∼60× below the cheapest proprietary one ($2.62). Two reasons make this decisive: regulated SOCs cannot send telemetry to a third-party API, and even where allowed, frontier per-evaluation cost compounds across analyst workloads. A 32B local checkpoint that closes most of the gap changes the calculus from "frontier or nothing" to "frontier when allowed, on-premises 32B otherwise." I think the results look terrific!

## VII. Open Challenges and Conclusion

Several directions follow from the v21 results. The most direct is layering a **reinforcement-learning phase with verifiable CTI rewards** on top of the IFT chain—essentially a Stage 5 "Verify"—scoring completions against Ariadne-derived gold answers for closed-form tasks (RCM, VSP) or rubrics for open-ended ones (RMS, TAA). Minerva [30] (by a co-author) shows that RL with verifiable CTI rewards delivers gains pure IFT leaves on the table, echoing the RL-style refinement in SecKnowledge 2.0 [6]. A second direction is **larger base models** (70–72B) to mitigate the parameter-count-graded general-knowledge erosion documented above, at roughly 2× the serving footprint. Further directions include **embedding-similarity dedup** for paraphrastic leakage beyond the n-gram guard, **hybrid template/LLM generation** for surface diversity while preserving correctness, **CoT enrichment** of Sophia rows [6], and **operational evaluation** (prompt-injection robustness, latency, SIEM/SOAR integration) [2], [16], [18].

We address contamination explicitly. The v21 build distinguishes *verbatim* contamination (eval text reproduced in a training row—bad) from *structural* overlap (training and eval rows independently derived from the same public graph—accepted). A single guard, `dedup_against_evals.py`, runs once per shard at build time: it indexes the union of $n$=13 word-grams across every AthenaBench/CTIBench/CyberMetric/CyberSOCEval eval file, then flags any SFT row sharing a 13-gram (`--hit-threshold1`) and filters any row reaching 50 shared 13-grams against a single eval row (`--drop-threshold50`)—the same window used by OLMo/Pythia/Llama decontamination. Paraphrastic leakage, which n-gram dedup misses, is addressed structurally: Sophia templates are generated from the graph, not from eval text, so overlap is only possible when both rows independently describe the same graph fact. The guard is enforced per shard (a non-zero exit skips dataset emission), and all five architecture ports consume the same deduped shards. ref. available for this?

In summary, chained IFT refines standard IFT along graph-grounded data construction and multi-stage scheduling, and Glaukopis instantiates it for CTI. The v21 ship adds 7.3 Avg-CTI-Benchmark and 13.3 Avg-CTI-Test points over a strong open base without changing parameter count, approaches frontier-class CTI performance within 4–7 points at ∼two orders of magnitude lower per-evaluation cost, and remains auditable and incrementally refreshable: every row is materialized by a typed Cypher query traceable to public-source keys, and every stage's contribution is independently measurable. For regulated SOC environments where third-party APIs cannot be deployed, disciplined chained IFT on an on-premises 32B base is a viable alternative to frontier proprietary models.

## Data and Funding

This research was funded by Athena Security Group, LLC, which develops the Glaukopis system; no other competing interests are declared. Ariadne is built from twelve independently accessible public CTI sources (Section IV). The Ariadne populator, the Sophia generator, the v21 manifests, the IFT launchers, and the evaluation harness are released at `https://github.com/`

TABLE II

HEADLINE RESULTS: GLAUKOPIS V21 SHIP CHECKPOINT VS. BASE MODEL AND FRONTIER REFERENCE POINTS. THREE AVERAGES ARE REPORTED PER THE SCHEMES DEFINED IN SECTION VI: **AVG CTI BENCH** (3 BENCHMARKS, EQUAL WEIGHT), **AVG CTI TEST** (10 CTI TESTS, EQUAL WEIGHT), AND **AVG TOTAL (W/MMLU)** (10 CTI TESTS PLUS MMLU-PRO). ALL NUMBERS FROM THE 2026-05-24 EVALUATION SWEEP. AB AVG II IS THE ATHENABENCH COMPOSITE USING THE 50/50 TAA CLASSIC+CANONICAL BLEND.

| Model | Type | Avg CTI Bench | Avg CTI Test | Avg Total (w/MMLU) | AB Avg II | CSE comb | CM avg | MMLU-Pro |
|---|---|---|---|---|---|---|---|---|
| GPT-5.5 | Frontier | 72.9 | 73.5 | 76.9 | 77.0 | 48.3 | 93.3 | 88.3 |
| DeepSeek-V4-Pro | OS-PT | 70.4 | 67.9 | 69.6 | 67.2 | 52.2 | 91.8 | 79.1 |
| Gemini-3-Flash-Preview | Frontier | 69.8 | 68.4 | 71.7 | 69.3 | 48.1 | 92.1 | 89.2 |
| GPT-5.2 (reasoning=medium) | Frontier | 67.5 | 65.0 | 67.0 | 63.8 | 46.8 | 91.8 | 81.3 |
| DeepSeek-v3.2-Exp | OS-PT | 66.0 | 62.2 | 63.9 | 59.0 | 47.2 | 91.7 | 82.5 |
| **v21-recal-32b (ship)** | SFT | **66.3** | **62.9** | **62.8** | **66.5** | 42.6 | 89.8 | 57.4 |
| v21-cse | SFT | 65.8 | 62.4 | 62.2 | 66.6 | 42.0 | 88.8 | 54.2 |
| v21-recalibrate (14B-recipe) | SFT | 65.9 | 62.4 | 62.2 | 65.9 | 42.4 | 89.2 | 54.6 |
| Gemini-2.5-Flash | Frontier | 63.4 | 57.9 | 59.8 | 53.1 | 46.2 | 91.0 | 82.1 |
| v21-core | SFT | 62.6 | 61.6 | 63.2 | 67.2 | 31.1 | 89.6 | 59.2 |
| v21-taa | SFT | 62.0 | 61.7 | 63.7 | 66.9 | 29.8 | 89.4 | 58.2 |
| Qwen2.5-32B-Instruct (base) | Instruct | 59.0 | 49.6 | 49.3 | 44.2 | 42.6 | 90.2 | 69.0 |

TABLE III

PER-STAGE ATHENABENCH BREAKDOWN FOR THE QWEN-2.5-32B V21 CHAIN (COMBINED-FORM SCORES WHERE MULTIPLE FORMS ARE RELEASED). STAGE 1 (CORE) DOES MOST OF THE ATHENABENCH-CATALOG WORK; STAGE 2 INTRODUCES CANONICAL TAA; STAGE 3 LIFTS CYBERSOCEVAL; STAGE 4 RECOVERS VSP WITHOUT LOSING PRIOR GAINS.

| Checkpoint | CKT | ATE | RCM | RMS | VSP | TAA Cl. | TAA Can. | AB II |
|---|---|---|---|---|---|---|---|---|
| Qwen2.5-32B-Instruct (base) | 52.9 | 18.2 | 59.5 | 7.6 | 78.8 | 48.0 | 2.0 | 44.2 |
| v21-core | 72.9 | 67.2 | 69.8 | 63.8 | 83.1 | 46.5 | 18.4 | 67.2 |
| v21-taa | 75.4 | 63.4 | 69.0 | 62.9 | 83.5 | 47.0 | 36.8 | 66.9 |
| v21-cse | 72.5 | 63.4 | 67.7 | 63.7 | 78.9 | 53.5 | 3.4 | 66.6 |
| **v21-recal-32b** | 75.5 | 65.8 | 67.1 | 62.5 | 77.8 | 50.5 | 3.4 | **66.5** |

TABLE IV

CROSS-ARCHITECTURE V21 PORTS. *Best v21*: HIGHEST-SCORING SFT CHECKPOINT FOR THAT BASE ARCHITECTURE ON AVG CTI BENCHMARK SCORE. **AVG CTI BENCH** AND **AVG CTI TEST** ARE THE PER-BENCHMARK AND PER-TEST SCHEMES DEFINED IN SECTION VI. $\Delta$ *test* AND $\Delta$ *MMLU*: ABSOLUTE DIFFERENCES BETWEEN THE BEST SFT CHECKPOINT AND THE CORRESPONDING INSTRUCT BASE ON AVG CTI TEST AND MMLU-PRO RESPECTIVELY. THE QWEN-2.5-32B DENSE PORT IS THE CROSS-ARCHITECTURE OPTIMUM ON CTI; THE QWEN3-MOE PORT LEADS ON MMLU-PRO AMONG SFT'D CHECKPOINTS BUT IS MORE EXPENSIVE TO SERVE AND BENCHMARK.

| Base architecture | Best v21 checkpoint | Avg CTI Bench | Avg CTI Test | $\Delta$ test vs. base | MMLU-Pro | $\Delta$ MMLU vs. base |
|---|---|---|---|---|---|---|
| Qwen2.5-32B-Instruct | v21-recal-32b | **66.3** | **62.9** | +13.3 | 57.4 | −11.6 |
| Qwen2.5-14B-Instruct | v21-recalibrate | 62.3 | 59.6 | +13.5 | 50.2 | −13.9 |
| Qwen3-30B-A3B-Thinking-2507 | v21-cse | 63.4 | 60.9 | +6.0 | 53.0 | −24.6 |
| Foundation-Sec-8B-Instruct | v21-recalibrate | 53.8 | 53.2 | +0.4 | 25.6 | −17.6 |
| Llama-3.1-8B-Instruct | v21-recalibrate | 50.5 | 50.8 | +7.3 | 27.7 | −20.6 |

`Athena-Software-Group/Glaukopis` [8]; cumulative checkpoints (`asg-ai/athena-cti-sft-qwen25-32b-v21-\{core,taa,cse,recal-32b\}` and cross-architecture ports) are on the Hugging Face Hub.

## REFERENCES


[1] M. Levi, Y. Allouche, D. Ohayon, and A. Puzanov, "Cyberpal.ai: Empowering LLMs with expert-driven cybersecurity instructions," *arXiv preprint arXiv:2408.09304*, 2024. [Online]. Available: https://arxiv.org/abs/2408.09304

[2] Z. Liu, J. Shi, and J. F. Buford, "Cyberbench: A multi-task benchmark for evaluating large language models in cybersecurity," *arXiv preprint arXiv:2407.08665*, 2024. [Online]. Available: https://arxiv.org/abs/2407.08665

[3] J. Wei, M. Bosma, V. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," in *International Conference on Learning Representations (ICLR)*, 2022. [Online]. Available: https://arxiv.org/abs/2109.01652

[4] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," *arXiv preprint arXiv:2203.02155*, 2022. [Online]. Available: https://arxiv.org/abs/2203.02155

[5] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, E. Li, X. Wang, M. Dehghani, S. Brahma *et al.*, "Scaling instruction-finetuned language models," *arXiv preprint arXiv:2210.11416*, 2022. [Online]. Available: https://arxiv.org/abs/2210.11416

[6] M. Levi, D. Ohayon, A. Blobstein, R. Sagi, I. Molloy, and Y. Allouche, "Toward cybersecurity-expert small language models," *arXiv preprint arXiv:2510.14113*, 2025. [Online]. Available: https://arxiv.org/abs/2510.14113

[7] H. Ji, J. Yang, L. Chai, C. Wei, L. Yang, Y. Duan, Y. Wang, T. Sun, H. Guo, T. Li *et al.*, "Sevenllm: Benchmarking, eliciting, and enhancing abilities of large language models in cyber threat intelligence," *arXiv preprint arXiv:2405.03446*, 2024. [Online]. Available: https://arxiv.org/abs/2405.03446

[8] Athena Security Group, "Glaukopis: An open-source implementation of chained instruction fine-tuning for CTI expert LLMs," https://github.

TABLE V

FULL-SUITE EVALUATION COST ($USD) FOR THE V21 SHIP CHECKPOINT VS. FRONTIER REFERENCE POINTS AND GLAUKOPIS INTERMEDIATE STAGES. COST IS THE SUM OF PER-BENCHMARK COST ACROSS ATHENABENCH, CYBERSOCEVAL, CYBERMETRIC 2K+10K, AND MMLU-PRO. LOCAL CHECKPOINTS ARE BILLED AT $2.50/GPU-HR ON 2×H100; HOSTED-API CHECKPOINTS ARE PRICED BY PUBLISHED PER-1K-TOKEN RATE. *Cost ratio* IS FULL-SUITE COST DIVIDED BY THE V21 SHIP COST ($2.83); *Avg CTI Bench* IS REPRODUCED FROM TABLE II FOR COST-VS-CAPABILITY CONTEXT.

| **Model** | **Billing basis** | **Full-suite cost (USD)** | **Cost ratio** | **Avg CTI Bench** | **$/CTI point** |
|---|---|---|---|---|---|
| *Frontier proprietary (API-priced)* | | | | | |
| Gemini-3-Flash-Preview | API tokens | 1,154.13 | 408× | 69.8 | 16.5 |
| GPT-5.5 (reasoning=medium) | API tokens | 508.13 | 180× | 72.9 | 6.97 |
| GPT-5.2 (reasoning=medium) | API tokens | 209.67 | 74× | 67.5 | 3.11 |
| Gemini-2.5-Flash | API tokens | 166.24 | 59× | 63.4 | 2.62 |
| *Frontier open-weights (API-priced)* | | | | | |
| DeepSeek-V4-Pro | API tokens | 126.40 | 45× | 70.4 | 1.80 |
| DeepSeek-V3.1-Terminus | API tokens | 14.33 | 5× | — | — |
| DeepSeek-V3.2-Exp | API tokens | 11.24 | 4× | 66.0 | 0.17 |
| *Glaukopis 32B and base (locally served)* | | | | | |
| Qwen2.5-32B-Instruct (base) | 2×H100 | 10.67 | 3.8× | 59.0 | 0.18 |
| **v21-recal-32b (ship)** | 2×H100 | **2.83** | **1.0×** | **66.3** | **0.043** |
| v21-recalibrate (14B-recipe) | 2×H100 | 2.51 | 0.9× | 65.9 | 0.038 |
| v21-cse | 2×H100 | 2.18 | 0.8× | 65.8 | 0.033 |
| v21-core | 2×H100 | 2.88 | 1.0× | 62.6 | 0.046 |
| v21-taa | 2×H100 | 2.73 | 1.0× | 62.0 | 0.044 |


com/Athena-Software-Group/Glaukopis, 2026, gitHub repository.

[9] S. Gururangan, A. Marasović, S. Swayamdipta, K. Lo, I. Beltagy, D. Downey, and N. A. Smith, "Don't stop pretraining: Adapt language models to domains and tasks," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 8342–8360. [Online]. Available: https://arxiv.org/abs/2004.10964

[10] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," *arXiv preprint arXiv:2201.11903*, 2022. [Online]. Available: https://arxiv.org/abs/2201.11903

[11] C. Fieblinger, A. M. Tjoa *et al.*, "Actionable cyber threat intelligence using knowledge graphs and large language models," *arXiv preprint*, 2024.

[12] X. Wu, L. Wang *et al.*, "Open-cykg: Constructing a cyber threat intelligence knowledge graph from open sources," *arXiv preprint*, 2022.

[13] Y. Yu, J. Kang *et al.*, "Ctikg: A large-scale, high-quality cyber threat intelligence knowledge graph," *arXiv preprint*, 2024.

[14] Z. Zhao, H. Li *et al.*, "K-ctiaa: Knowledge graph enhanced cyber threat intelligence action analysis," *arXiv preprint*, 2024.

[15] Y. Zhang, H. Chen *et al.*, "Cyberkg: A cyber threat intelligence knowledge graph for threat analysis," *arXiv preprint*, 2023.

[16] M. T. Alam, D. Bhusal, S. Ahmad, N. Rastogi, and P. J. Worth, "Athenabench: A dynamic benchmark for evaluating LLMs in cyber threat intelligence," *arXiv preprint arXiv:2511.01144*, 2025. [Online]. Available: https://arxiv.org/abs/2511.01144

[17] N. Tihanyi, M. A. Ferrag, R. Jain, T. Bisztray, and M. Debbah, "CyberMetric: A benchmark dataset based on retrieval-augmented generation for evaluating LLMs in cybersecurity knowledge," *arXiv preprint arXiv:2402.07688*, 2024. [Online]. Available: https://arxiv.org/abs/2402.07688

[18] H. Hu, P. Zhang *et al.*, "Seceval: A comprehensive benchmark for evaluating cybersecurity knowledge in foundation models," *arXiv preprint*, 2024.

[19] M. Bhatt, S. Chennabasappa, C. Nikolaidis, S. Wan, I. Evtimov, D. Gabi, D. Song, F. Ahmad, C. Aschermann, L. Fontana, S. Frolov, R. P. Giri, D. Kapil, Y. Kozyrakis, D. LeBlanc, J. Milazzo, A. Straumann, G. Synnaeve, V. Vontimitta, S. Whitman, and J. Saxe, "Purple Llama CyberSecEval: A secure coding benchmark for language models," *arXiv preprint arXiv:2312.04724*, 2023. [Online]. Available: https://arxiv.org/abs/2312.04724

[20] M. Bhatt, S. Chennabasappa, Y. Li, C. Nikolaidis, D. Song, S. Wan, F. Ahmad, C. Aschermann, Y. Chen, D. Kapil, D. Molnar, S. Whitman, and J. Saxe, "CyberSecEval 2: A wide-ranging cybersecurity evaluation suite for large language models," *arXiv preprint arXiv:2404.13161*, 2024. [Online]. Available: https://arxiv.org/abs/2404.13161

[21] N. Li, A. Pan, A. Gopal, S. Yue, D. Berrios, A. Gatti, J. D. Li, A.-K. Dombrowski, S. Goel, L. Phan, G. Mukobi, N. Helm-Burger, R. Lababidi, L. Justen, A. B. Liu, M. Chen, I. Barrass, O. Zhang, X. Zhu, R. Tamirisa, B. Bharathi, A. Khoja, Z. Zhao, A. Herbert-Voss, C. B. Breuer, S. Marks, O. Patel, A. Zou, M. Mazeika, Z. Wang, P. Oswal, W. Lin, A. A. Hunt, J. Tienken-Harder, K. Y. Shih, K. Talley, J. Guan, R. Kaplan, I. Steneker, D. Campbell, B. Jokubaitis, A. Levinson, J. Wang, W. Qian, K. K. Karmakar, S. Basart, S. Fitz, M. Levine, P. Kumaraguru, U. Tupakula, V. Varadharajan, R. Wang, Y. Shoshitaishvili, J. Ba, K. M. Esvelt, A. Wang, and D. Hendrycks, "The WMDP benchmark: Measuring and reducing malicious use with unlearning," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. [Online]. Available: https://arxiv.org/abs/2403.03218

[22] M. T. Alam, D. Bhusal, N. Shah, and N. Rastogi, "Ctibench: A benchmark for evaluating LLMs in cyber threat intelligence," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. [Online]. Available: https://arxiv.org/abs/2406.07508

[23] Meta AI, "CyberSOCEval: Benchmarking LLMs' capabilities for malware analysis and threat intelligence reasoning," https://ai.meta.com/research/publications/cybersoceval-benchmarking-llms-capabilities-for-malware-analysis-and-threat-intelligence-reasoning/, 2025, meta AI research publication.

[24] I. Cardei, "Instruction fine tuning dataset design," Athena Labs, Athena Security Group, Tech. Rep., 2025, internal design document describing the Athena CTI Neo4j graph schema and template-based IFT dataset generation.

[25] R. McMillan, "Defining threat intelligence," Gartner, Tech. Rep., 2013.

[26] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and philosophy," The MITRE Corporation, Tech. Rep. MP180360, 2018. [Online]. Available: https://attack.mitre.org/resources/

[27] "Common attack pattern enumeration and classification (CAPEC)," https://capec.mitre.org/, accessed 2025-12-03.

[28] "Common weakness enumeration (CWE)," https://cwe.mitre.org/, accessed 2025-12-03.

[29] "Common vulnerabilities and exposures (CVE)," https://www.cve.org/, accessed 2025-12-03.

[30] M. T. Alam *et al.*, "Minerva: Reinforcement learning for cyber threat intelligence reasoning," *arXiv preprint arXiv:2602.00513*, 2026. [Online]. Available: https://arxiv.org/abs/2602.00513